

Assignment4

Info

Name: Cody (Zhexian) Liu

Assignment Number: 4

Repo Link: <https://github.com/Rorical/CWM-FDI>

Buffer Overflow Attacks – Hello Overflow!

2 & 3. Run with offsets and complete table

```
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 6
Target Function Ptr: 0x4011dc

ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 8
Target Function Ptr: 0x4011dc

ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 24
Target Function Ptr: 0x4011dc
Success! Malicious function Called!
Illegal instruction (core dumped)

ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 30
Target Function Ptr: 0x4011dc
Segmentation fault (core dumped)

ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 40
Target Function Ptr: 0x4011dc
Segmentation fault (core dumped)

ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 60
Target Function Ptr: 0x4011dc
Segmentation fault (core dumped)
```

Offset	Observed Output	Notes
8	Target Function Ptr: 0x4011dc	strcpy stops at null byte in address (0x4011dc → dc 11 40 00); no overflow
24	Target Function Ptr: 0x4011dc Success! Malicious function	Exactly overwrites return address with 0x4011dc

Offset	Observed Output	Notes
	Called! Illegal instruction	
30	Target Function Ptr: 0x4011dc Segmentation fault	Address written past return address into caller's stack frame
40	Target Function Ptr: 0x4011dc Segmentation fault	Same as above
60	Target Function Ptr: 0x4011dc Segmentation fault	Same

4. Why is 24 the magic number?

The stack layout inside `victim()` is:

```

Higher addresses
+-----+
|      return address      | 8 bytes    ← offset 24–31
+-----+
|      saved RBP           | 8 bytes    ← offset 16–23
+-----+
|      buf[16]             | 16 bytes   ← offset 0–15
+-----+
Lower addresses (RSP)

```

- `buf` is 16 bytes
- `saved RBP` is 8 bytes
- `return address` is 8 bytes

Observation: $24 - 16 = 8$ which is exactly the size of a pointer (8 bytes). This means the address goes across the 16 bytes of buffer, then another 8 bytes of address, and successfully overwrite the 8 bytes of return address.

5. Small vs large input

In `main.c`, `victim("0123456789ABCDE")` runs first with a 15-character string, so it returns normally. Then `victim(evil_str)` runs with the overflow payload.

Small input (<16B) `strcpy` copies the short string plus null terminator entirely within `buf[16]`. Nothing beyond the buffer is overwritten, so the function returns normally to `main()`

Large input (>100B) strcpy copies well past `buf[16]` , overwriting the saved `RBP` , the return address, and beyond. The return address is filled with `0x4141414141414141` , causing the CPU to jump to an invalid address, resulting in a segmentation fault.

6. Hexdump trace of `append_address`

```
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ make && ./main 0
Target Function Ptr: 0x4011dc
0: ef
1: be
2: ad
3: de
4: \0
```

Buf Offset	Address	Content (hex)
Buf[0]	0xdeadbeef byte 0	0xef
Buf[1]	0xdeadbeef byte 1	0xbe
Buf[2]	0xdeadbeef byte 2	0xad
Buf[3]	0xdeadbeef byte 3	0xde

The bytes are stored in **little-endian** order: the least significant byte (`0xef`) is stored first at the lowest address, and the most significant byte (`0xde`) is stored last. `0xdeadbeef` is written in big-endian which explains why the print output is reversed.

7. Why `strcpy` is the culprit and replacement

`strcpy(dest, src)` copies bytes until it encounters a null byte (`\0`), without any bounds checking. If `src` is longer than `dest` , it keeps writing past the end of `dest` , overwriting the stack memory next to it.

The safe replacement is `strncpy(dest, src, n)` . It copies at most `n` bytes, preventing buffer overflows when `n` is set to the size of `dest` .

Buffer Overflow Attacks – Mounting Your First Attack

9. Functions in `vuln.c`

Functions: `main` , `authenticate_user` , `validate_username` , `get_user_info` , `process_command` , `print_banner` , `win` . The target is `win()` which prints the flag.

11. Which function causes segfault?

`process_command()` uses the vulnerable `gets()` function to read into a 256-byte response buffer. Passing a large input (>256 bytes) causes a buffer overflow and let us rewrite memory nearby, then invoke `segfault`.

12. Offset that causes segfault

Running `exploit.py` reveals the crash at **264 bytes**:

```
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4B$ python3 exploit.py
...
264          | -7          | CRASHED! (SIGSEGV)
[!] Potential offset found at: 264 bytes
```

13. Exploit and flag

`exploit.py` reports:

- `win()` address: **0x401216**
- `main()` address: **0x401679**

Updated `exploit.py`:

```
target_address = 0x401216 # win() function address
offset = 264             # offset that overwrites return address
```

Re-ran the exploit:

```
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4B$ python3 exploit.py
...
=====
ACCESS GRANTED
=====
FLAG: CTF{S3CURITY_CWM_WIN_2_EZ}
```

By parsing ELF table in `exploit.py` we get the address of the `win` function (`0x401216`) which we want to execute. The vulnerability exists in `process_command` function so we need to hijack it using buffer overflow, so we overwrite the return address to `0x401216` . When it return, it will invoke the `win` function and print the flag. Response buffer has size of 256, and add the 8 bytes of saved RBP address, we obtained a offset of 264. This offset push the address exactly to the return memory pointer. The `exploit.py` use fuzz testing to reveal the offset but in this simple case we can calculate it manually.

Timing Attacks

18. Why a correct leading character takes longer

From `views.py`, the `_vulnerable_check` function:

```
for i, ch in enumerate(password):
    if i >= len(SECRET_PASSWORD) or ch != SECRET_PASSWORD[i]:
        return False
    time.sleep(DELAY_PER_CHAR)
```

The code only returns `False` on the first mismatch. For each correct character, it calls `time.sleep(DELAY_PER_CHAR)`. A guess with *no* correct leading characters returns `False` immediately on the first character check, without sleeping. In other word, if success sequence is longer, it takes more time to return, which gives us hint about whether we are correct for a password prefix.

19. Why this is a problem despite identical error messages

A malicious attacker could use side-channel attack. Instead of seeing page content, attacker can measure the time it takes for the web server to respond. This is indirect and hard to find, but gives valuable information. To mitigate fluctuation of time when doing measurements, attacker can repeat hundreds of times and get average for the same request.

20. Measure single-character guesses

Guess	Time (ms)
0	1.7
1	1.8
2	1.9
3	1.8
4	1.8
5	3.9
6	1.7
7	1.8
8	1.8
9	1.7

21. Which guess took longer?

Guess "5" took 3.9 ms, noticeably longer than all other attempts. This tells us the first character of the password is **5**.

22. Extract the second character

Guess	Time (ms)
50	4.2
51	6.5
52	4.5
53	4.5
54	4.6
55	4.6
56	4.5
57	5.1
58	4.5
59	4.5

The longest time was **"51"** at 6.5ms. The first two characters are **51**.

25. Cracked password and flag

The cracked password is **519265** and the captured flag is **CWM{t1m1ng_1s_3v3ryth1ng}**.

```

'51926y'          15.4 ms
'51926z'          15.6 ms
'519260'          15.7 ms
'519261'          15.5 ms
'519262'          15.9 ms
'519263'          15.5 ms
'519264'          15.5 ms
'519265'          17.7 ms
'519266'          15.7 ms
'519267'          15.7 ms
'519268'          15.4 ms
'519269'          15.5 ms

[+] prefix so far: '519265'

[*] Password cracked: '519265'
<!DOCTYPE html>
<html>
<head><title>Login</title>
<style>
  body { font-family: monospace; max-width: 480px; margin: 60px auto; }
  .result { margin-top: 16px; padding: 12px; border: 1px solid #ccc; }
  .error { color: #c00; }
  .ok { color: #080; }
  .time { font-size: 1.4em; font-weight: bold; }
</style>
</head>
<body>
  <h2>Login</h2>

  <div class="result ok">
    <p>Access granted. Flag: <strong>CWM{t1m1ng_1s_3v3ryth1ng}</strong></p>
    <p>Check took: <span class="time">12.9 ms</span></p>
  </div>

  <form method="post" style="margin-top:24px">
    <input type="hidden" name="csrfmiddlewaretoken" value="1tTi1s2Lb0zdQZ0kF3p7tCPcPpNSF1NZ74WZshgIDsdV4u01qUun6I252Bp1wwH6">
    <label>Username<br><input type="text" name="username" value="admin"></label><br><br>
    <label>Password<br><input type="text" name="password" autocomplete="off"></label><br><br>
    <button type="submit">Try</button>
  </form>
</body>
</html>

```

26. Replacement safe_check

```

def safe_check(username: str, password: str) -> bool:
    """
    Constant-time password check that eliminates the timing side channel.
    Always iterates over all characters and sleeps the same amount per
    character,
    regardless of whether they match.
    """
    if username != SECRET_USERNAME:
        return False

    result = True
    max_len = max(len(password), len(SECRET_PASSWORD))

    for i in range(max_len):
        if i >= len(SECRET_PASSWORD) or i >= len(password) or password[i] !=
SECRET_PASSWORD[i]:
            result = False
            time.sleep(DELAY_PER_CHAR)

```

```
return result
```

This eliminates the timing side channel because:

- It always iterates `max_len` times regardless of where the first mismatch occurs
- It always sleep exactly `max_len` times
- The total execution time depends only on the length of the inputs, not on how many leading characters are correct
- The loop does not short-circuit or return early

27. Using third-party APIs in security-critical settings

1. **Never assume safety:** Even well-known, widely-used functions/libraries may have hidden vulnerabilities. Every dependency is a potential attack surface.
2. **Read the documentation for caveats:** Functions like `strcpy` and `gets` are perfectly documented as unsafe, but one must read `man` pages carefully to learn this.
3. **Audit dependencies for side channels:** Even if an API returns the correct output, it may leak information through timing, error messages, power consumption, or other side channels.
4. **Treat the system as a whole:** Security is not just about what a function returns but how it behaves.